

HabitGrid

Habit Tracker — Android Application

Software Requirements Specification

Version 1.0

Project Name	HabitGrid
Document Type	Software Requirements Specification (SRS)
Version	1.0
Platform	Android (React Native)
Prepared By	Ashwamedha
Status	Draft
Last Updated	23 June 2026

1. Introduction

1.1 Purpose

This document defines the complete software requirements for HabitGrid, an Android habit-tracking application built with React Native. It serves as the single reference point for the development team, covering every feature, behaviour, data model, and constraint that the system must satisfy.

1.2 Scope

HabitGrid lets users build and maintain daily habits through a calendar-grid interface inspired by the Google Calendar Android widget. Core capabilities include:

- Creating, editing, and deleting habits with rich metadata.
- Visualising habit history on a monthly calendar grid — one habit at a time, navigated with arrow controls.
- Long-pressing any date cell inside the app or on the home-screen widget to mark or unmark that day.
- Tracking streaks with a user-configurable grace period.
- Sending reminder notifications at user-defined times per habit.
- Storing all data locally by default, with an optional cloud-backup path for a future release.

1.3 Definitions

Habit	A recurring behaviour the user wants to track (e.g., "Read 20 pages").
Mark / Log	Recording that a habit was completed on a specific date.
Calendar Grid	The 7-column, week-row monthly view that displays completion data as coloured bars.
Widget	An Android App Widget placed on the device home screen that mirrors the in-app grid view.
Streak	The count of consecutive days on which a habit was marked, within the defined grace period.
Grace Period	The number of consecutive missed days allowed before a streak resets to zero.
Habit Switcher	The < > navigation arrows that move between habits in the grid view.

1.4 Document Overview

Section 2 describes the product from a high level. Section 3 lists all functional requirements. Section 4 covers non-functional requirements. Section 5 defines data models. Section 6 summarises the UI navigation flow. Section 7 lists external dependencies.

2. Overall Description

2.1 Product Perspective

HabitGrid is a standalone Android application. It is not a component of any larger system. The home-screen widget is a lightweight extension of the app that reads from the same local database. Cloud backup, when implemented, will be treated as an optional add-on that does not change the core offline experience.

2.2 Product Features — Summary

Habit Management	Create, edit, delete, and reorder habits. Each habit stores a title, colour, frequency, reminder time, notes, and a grace period.
Calendar Grid View	Monthly calendar showing one habit at a time. Coloured bars span dates where the habit was completed. Navigation arrows switch between habits.
Home-Screen Widget	App Widget (Android) replicating the grid view. Supports long-press to mark/unmark. Tapping elsewhere opens the app.
Marking System	Long press on any date cell — in-app or on the widget — toggles the completion state for that date.
Streak Engine	Calculates and displays the current streak and longest streak per habit. Grace period is configurable per habit.
Notifications	Daily reminder at a user-set time per habit. Notifications are cancellable and re-schedulable.
Data Storage	Local-first using SQLite via react-native-sqlite-storage. Cloud backup (Google Drive / custom API) is an optional future release.

2.3 User Classes

- Primary User — An individual adult or student who wants to build personal habits. Technically average; must not need a manual to use core features.
- Future Admin (out of scope v1) — If multi-device sync is introduced later, an account owner may manage their own cloud data.

2.4 Operating Environment

- Platform: Android 8.0 (API 26) and above.
- Framework: React Native (latest stable) with Expo or bare workflow.
- Widget: android.appwidget.AppWidgetProvider via a native module or react-native-android-widget.
- Database: SQLite via react-native-sqlite-storage.
- Notifications: react-native-push-notification or @notifee/react-native.

2.5 Design Constraints

- The app must function fully offline. No feature may require an internet connection except optional cloud backup.
- The home-screen widget must refresh its data within 3 seconds of a mark/unmark action.
- The calendar grid must show a full 7-day-wide month view — it must not truncate weeks.
- Habit colours must meet a minimum contrast ratio of 4.5:1 against the grid cell background.

3. Functional Requirements

3.1 Habit Management

ID	Feature	Description	Priority
FR-01	Create Habit	User fills a form with title (required), colour picker, target frequency, reminder time, notes, and grace period. Habit is saved locally on submit.	High
FR-02	Edit Habit	User taps an edit icon on the habit detail screen to update any field. Changes persist immediately.	High
FR-03	Delete Habit	User deletes a habit after confirming a prompt. All associated log entries and streak data are also removed.	High
FR-04	Habit List Screen	A dedicated screen lists all habits with their colour swatch, title, current streak, and a quick-action menu. Reordering by drag is supported.	High
FR-05	Habit Colour Picker	A visual colour palette (minimum 12 distinct colours) lets the user assign a colour. The chosen colour renders as the bar fill in the grid.	Medium
FR-06	Target Frequency	User sets daily, weekly (pick days), or a custom interval. The streak engine uses this schedule to evaluate missed days.	Medium
FR-07	Notes Field	A multi-line text field on the habit form stores a short description or motivation note. Displayed on the habit detail screen.	Low

3.2 Calendar Grid View (In-App)

ID	Feature	Description	Priority
FR-08	Monthly Grid Display	The grid shows a standard calendar layout: 7 columns (Sun–Sat), 4–6 rows per month. Date numbers are visible in each cell.	High

FR-09	Completion Bars	Dates on which the current habit was marked show a solid coloured bar (same colour as the habit). Unmarked dates show no fill.	High
FR-10	Habit Name in Header	The currently displayed habit's title appears where the month name sits in the Google Calendar widget. Month and year appear below it.	High
FR-11	Habit Switcher Arrows	Tapping < moves to the previous habit in the list; tapping > moves to the next. The view wraps around at both ends.	High
FR-12	Month Navigation	A second pair of arrows (or swipe gesture) navigates between months for the same habit.	Medium
FR-13	Long Press to Mark	Long-pressing a date cell toggles the habit completion for that date. A brief haptic or visual feedback confirms the action.	High
FR-14	Today Highlight	Today's date cell has a distinct ring or fill to make it immediately identifiable.	Medium
FR-15	Add Habit Shortcut	A + button in the grid header opens the Create Habit form directly.	Medium

3.3 Home-Screen Widget

ID	Feature	Description	Priority
FR-16	Widget Layout	The widget replicates the in-app grid: habit title in the header, < > arrows, calendar cells, and coloured bars — displayed as an App Widget.	High
FR-17	Widget Long Press Mark	Long-pressing a date cell on the widget toggles the completion state and triggers a widget refresh. The app does not need to be open.	High
FR-18	Widget Habit Switcher	The < > arrows on the widget switch between habits. State (current habit index) is stored in SharedPreferences so it survives reboots.	High
FR-19	Widget Tap-to-Open	Tapping the widget's header or any non-interactive area opens the app to the corresponding habit's grid screen.	Medium
FR-20	Widget Auto-Refresh	The widget refreshes its data automatically when: (a) a mark is made in-app, (b) a new habit is created, or (c) on system boot.	High
FR-21	Widget + Button	A + button on the widget opens the Create Habit form inside the app.	Low

3.4 Streak Engine

ID	Feature	Description	Priority
FR-22	Current Streak	Displays the number of consecutive scheduled days (within grace period) the habit has been completed, counted backwards from today.	High
FR-23	Longest Streak	Stores and displays the all-time longest streak achieved for each habit.	Medium
FR-24	Grace Period	User sets a per-habit grace period (0–7 days). Missing up to that many consecutive scheduled days does not break the streak.	High
FR-25	Streak Display	Current streak and longest streak are shown on the habit detail screen and the habit list card.	Medium
FR-26	Retroactive Mark Fix	If a user marks a past date that was within an active grace period, the streak is recalculated immediately.	Medium

3.5 Reminders and Notifications

ID	Feature	Description	Priority
FR-27	Reminder Time	User sets a daily reminder time per habit. The app schedules a local notification at that time each day the habit is due.	High
FR-28	Notification Content	Notification title = habit name. Body = a short motivational nudge (configurable or random from a small built-in set).	Medium
FR-29	Enable / Disable	User can toggle reminders on or off per habit without losing the saved time.	High
FR-30	Deep-Link on Tap	Tapping a notification opens the app directly to that habit's grid view.	Medium

3.6 Data Storage and Backup

ID	Feature	Description	Priority
FR-31	Local Storage	All habit data and log entries are stored in a local SQLite database. No internet connection is needed.	High
FR-32	Data Persistence	Data survives app restarts, OS updates, and device reboots.	High
FR-33	Cloud Backup (Future)	A future version will allow authenticated users to back up their data to a cloud endpoint (Google Drive or custom API). The local schema must be designed to support this without a migration break.	Low

FR-34	Export (Future)	A future version will support exporting habit logs as CSV. No action required in v1.0 beyond keeping the data schema clean.	Low
-------	-----------------	---	-----

4. Non-Functional Requirements

4.1 Performance

- The calendar grid must render and respond to navigation within 300 ms on mid-range devices (2 GB RAM, Android 8).
- Marking a habit (long press) must update the UI within 200 ms.
- The home-screen widget must complete a data refresh and redraw within 3 seconds.
- App cold-start time must not exceed 3 seconds on the target hardware class.

4.2 Usability

- First-time users must be able to create their first habit and mark it on the calendar within 2 minutes, with no guidance.
- Habit switcher arrows must have a minimum tap target of 48 dp to comply with Android accessibility guidelines.
- Colour choices in the picker must include at least two options that are distinguishable for the most common forms of colour vision deficiency.

4.3 Reliability

- No habit log entry may be silently lost. Any write failure must surface a user-visible error and offer a retry.
- The streak engine must produce consistent results regardless of the timezone the device is in or changes to it.

4.4 Maintainability

- The codebase must separate the streak calculation logic into a standalone, unit-testable module.
- The SQLite schema must be versioned with a migration table to support future schema changes without data loss.

4.5 Security

- No personal data is transmitted to any server in v1.0.
- When cloud backup is introduced, all data must be transmitted over HTTPS and stored encrypted at rest.

5. Data Models

All tables are stored in the local SQLite database. Field names use snake_case.

5.1 Habit

Field	Type	Description
id	INTEGER PK	Auto-incremented primary key.
title	TEXT	Habit name displayed in the grid header and list. Required, max 60 chars.
color_hex	TEXT	6-character hex colour (e.g. 2D6BE4) used for the completion bar fill.
frequency_type	TEXT	daily weekly custom — controls which days the streak engine monitors.
frequency_days	TEXT	JSON array of ISO weekday numbers (0–6) for weekly/custom types. Null for daily.
reminder_time	TEXT	HH:MM string in 24-hour format. Null if reminders are off.
reminder_enabled	INTEGER	Boolean (0/1). Allows toggling notifications without clearing reminder_time.
notes	TEXT	Optional free-text description or motivation note. Max 280 chars.
grace_period	INTEGER	Number of consecutive missed scheduled days before streak resets. 0–7.
sort_order	INTEGER	User-defined display order in the habit list and grid switcher.
created_at	INTEGER	Unix timestamp (ms) of creation.

5.2 HabitLog

Field	Type	Description
id	INTEGER PK	Auto-incremented primary key.
habit_id	INTEGER FK	References Habit.id. Cascade delete.
log_date	TEXT	ISO 8601 date string (YYYY-MM-DD). One row per marked date per habit.
marked_at	INTEGER	Unix timestamp (ms) when the mark was created or last toggled.

5.3 StreakCache

Computed fields stored to avoid recalculating on every render. Invalidated on any HabitLog insert, update, or delete for the relevant habit.

Field	Type	Description
habit_id	INTEGER PK FK	One row per habit. References Habit.id.
current_streak	INTEGER	Consecutive completed days (within grace period) as of today.
longest_streak	INTEGER	All-time maximum streak achieved.
last_computed	INTEGER	Unix timestamp (ms) of last recalculation.

6. UI Navigation Flow

The app has four primary screens. Navigation is stack-based with a persistent bottom tab between the Grid View and the Habit List.

Splash / Init Screen	Loads the SQLite database, applies any pending migrations, then redirects to the Grid View.
Grid View (Home)	Shows the calendar grid for the active habit. Header: habit title, < > arrows, + button, month/year. Body: 7-col grid with coloured bars on marked dates. Long press a cell to toggle. Swipe left/right to change month.
Habit List Screen	Full list of all habits. Each card shows colour swatch, title, current streak, longest streak, and a three-dot menu (Edit / Delete). Drag handle for reorder. FAB to add a new habit.
Create / Edit Habit	A modal or pushed screen with form fields: title, colour picker, frequency selector, reminder time picker, notes textarea, grace period stepper. Save and Cancel buttons.
Habit Detail (future)	Tapping a habit in the list could open a detail view showing full-month streak visualisation. Out of scope for v1.0.

7. External Dependencies

Package	Purpose	Notes
react-native-sqlite-storage	Local database	Provides the SQLite driver. Must be linked via autolinking.
react-native-android-widget	Home-screen widget	Enables React-rendered App Widgets on Android 8+.
@notifee/react-native	Local notifications	Schedules and cancels daily reminder notifications.
react-native-haptic-feedback	Mark confirmation	Delivers haptic feedback on long-press mark actions.
react-native-async-storage	Widget state	Stores current habit index and widget config in SharedPreferences.
date-fns	Date arithmetic	Used by the streak engine for reliable date calculations across timezones.

8. Open Items for v1.0

The following decisions are deferred and must be resolved before the corresponding module is built.

1. Cloud backup provider: Google Drive OAuth vs a custom REST API. Decision needed before FR-33 is started.
2. Widget long-press implementation: react-native-android-widget's long-press support must be verified against the target API levels. A fallback (tap-to-open the app) should be planned if native support is unavailable on API 26–28.
3. Habit reorder persistence: confirm whether `sort_order` is updated on every drag-end or only on a save action.
4. Maximum habit count: no limit is specified. A soft cap (e.g. 20 habits) should be considered to keep widget performance predictable.

End of Document — HabitGrid SRS v1.0